

BARE METAL / SHARP X68000

PONG

Human68k非依存のC実装
GameModeでタイトルループ
デモを追加

elf2x68k + IOCS + mode 12

Makefile -> dist/pong.xdf



DemoStateをGameContextへ追加して責務を分離

モード固有状態

TitleState

selected / idle_frames

DemoState

elapsed_frames

WinnerState

player / start

GameContext

application : ApplicationState

title : TitleState

pong : PongGameState

demo : DemoState

winner : WinnerState

状態の所有先を明確化

makeだけでベアメタルXDFまで生成

01 /
COMPILE

m68k-xelf-gcc

-m68000 -O2
-specs=x68knodos.specs
リンク先 0x6800

02 /
BOOT

human.sysへ配置

リンク時にhuman.sysを直接生成。
Human68k本体やシステムファイル
は不要。

03 / XDF

dist/pong.xdf

human.sysだけを格納した
新しいXDFイメージを生成。

512 x 480 / 60 Hz表示

MODE

SYNC

TIMING

CRTMOD(12)

512 x 512 / 65536色の
標準31 kHzモードを選択。

wait_vdisp()

MFP GPIPのV-DISPエッジを待ち
、
1更新を1フレームに固定。

R05/R06/R07 -> R04

表示位置を先に設定し、
最後に総走査線525へ変更。

_ONTIMEで待機タイムアウトを監視し、停止を回避

GAME_MODE_DEMOを状態遷移へ組み込む

```
typedef enum {  
    GAME_MODE_TITLE,  
    GAME_MODE_PONG,  
    GAME_MODE_DEMO,  
    GAME_MODE_WINNER,  
    GAME_MODE_EXIT  
} GameModeId;  
  
{demo_initialize,  
 demo_update,  
 demo_finalize}
```

GameModeの3関数を実装

- 01 initialize()で両CPU対戦を開始
- 02 update()の先頭で入力を確認
- 03 共通pong_game_update()を実行
- 04 15秒経過でTITLEを返す
- 05 finalize()で入力をクリア

TITLEは10秒無操作でDEMOへ遷移。

demo_update()は入力判定を最優先する

```
static GameModeId demo_update(...) {  
    if (wait_vdisp() != 0) return EXIT;  
    if (input_has_activity()) return TITLE;  
  
    pong_game_update(&context->pong, &input);  
    if (++elapsed >= 60 * 15) return TITLE;  
    return DEMO;  
}
```

INPUT FIRST

更新前に入力を確認

SHARED STEP

通常対戦と同じPONG更新を使用

10 SEC IDLE > DEMO > TITLE

共有処理と遷移を分けて変更範囲を限定

01 /
TITLE

02 /
DEMO

03 /
RETURN

IDLE

idle_framesを毎フレーム更新。
10秒無操作でDEMOへ。

PLAY

左右をCPUに設定し、共通の
pong_game_update()を実行。

INPUT / TIME

入力を検出、または15秒経過で
即座にTITLEへ戻る。

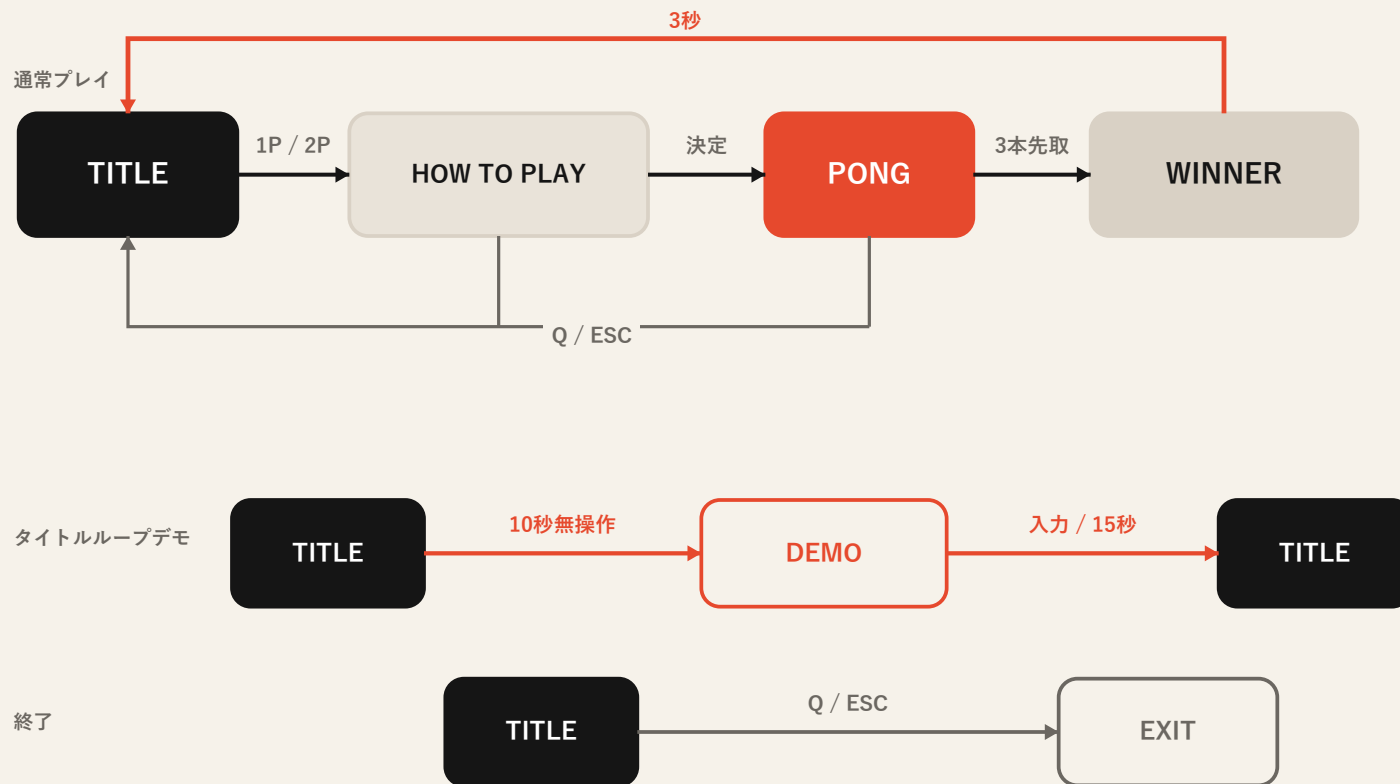
ゲーム本体は複製せず、GameModeだけを追加。

GameModeで画面と遷移を明示する

ゲームモード一覧

- **TITLE**
人数選択 / 無操作時間を計測
- **HOW_TO_PLAY**
操作説明 / 入力解放を待機
- **PONG**
対戦処理を共有 / 3本先取
- **WINNER**
勝者を表示 / 3秒保持
- **DEMO**
CPU対CPU / 15秒のデモ
- **EXIT**
画面モードを復元 / 終了

モード遷移



新しい画面は、状態と3つのライフサイクル関数 `initialize` / `update` / `finalize` を追加する。